第 12 章

并发程序的测试

在编写并发程序时，可以采用与编写串行程序时相同的设计原则与设计模式。二者的差异在于，并发程序存在一定程度的不确定性，而在串行程序中不存在这个问题。这种不确定性将增加不同交互模式以及故障模式的数量，因此在设计并发程序时必须对这些模式进行分析。

同样，在测试并发程序时，将使用并扩展许多在测试串行程序时用到的方法。在测试串行程序正确性与性能等方面所采用的技术，同样可以用于测试并发程序，但对于并发程序而言，可能出错的地方远比串行程序多。在测试并发程序时，所面临的主要挑战在于：潜在错误的发生并不具有确定性，而是随机的。要在测试中将这些故障暴露出来，就需要比普通的串行程序测试覆盖更广的范围并且执行更长的时间。

并发测试大致分为两类，即安全性测试与活跃性测试。在第 1 章，我们将安全性定义为“不发生任何错误的行为”，而将活跃性定义为“某个良好的行为终究会发生”。

在进行安全性测试时，通常会采用测试不变性条件的形式，即判断某个类的行为是否与其规范保持一致。例如，假设有一个链表，在它每次被修改时把其大小缓存下来，那其中一项安全性测试就是比较在缓存中保存的大小值与链表中实际元素的数目是否相等。这种测试在单线程程序中很简单，因为在测试时链表的内容不会发生变化。但在并发程序中，这种测试将可能由于竞争而失败，除非能将访问计数器的操作和统计元素数据的操作合并为单个原子操作。要实现这一点，可以对链表加锁以实现独占访问，然后采用链表中提供的某种“原子快照”功能，或者在某些“测试点 (Test Point)”上采用原子方式来判断不变性条件或者执行测试代码。

在本书中，我们曾通过执行时序图来说明“错误的”交互操作，这些操作将在未被正确构造的类中导致各种故障，而测试程序将努力在足够大的状态空间中查找出错的地方。然而，测试代码同样会对执行时序或同步操作带来影响，这些影响可能会掩盖一些本可以暴露的错误。[⊖]

测试活跃性本身也存在问题。活跃性测试包括进展测试和无进展测试两方面，这些都是很难量化的——如何验证某个方法是被阻塞了，而不只是运行缓慢？同样，如何测试某个算法不会发生死锁？要等待多久才能宣告它发生了故障？

与活跃性测试相关的是性能测试。性能可以通过多个方面来衡量，包括：

吞吐量：指一组并发任务中已完成任务所占的比例。

响应性：指请求从发出到完成之间的时间（也称为延迟）。

⊖ 这些在增加了调试代码或测试代码后消失的错误称为“海森堡错误”（Heisenbugs，来源于“海森堡测不准原理”）。

可伸缩性：指在增加更多资源的情况下（通常指 CPU），吞吐量（或者缓解短缺）的提升情况。

12.1 正确性测试

在为某个并发类设计单元测试时，首先需要执行与测试串行类时相同的分析——找出需要检查的不变性条件和后验条件。幸运的话，在类的规范中将给出其中大部分的条件，而在剩下的时间里，当编写测试时将不断地发现新的规范。

为了进一步说明，接下来我们将构建一组测试用例来测试一个有界缓存。程序清单 12-1 给出了 BoundedBuffer 的实现，其中使用 Semaphore 来实现缓存的有界属性和阻塞行为。

程序清单 12-1 基于信号量的有界缓存

```
@ThreadSafe
public class BoundedBuffer<E> {
    private final Semaphore availableItems, availableSpaces;
    @GuardedBy("this") private final E[] items;
    @GuardedBy("this") private int putPosition = 0, takePosition = 0;

    public BoundedBuffer(int capacity) {
        availableItems = new Semaphore(0);
        availableSpaces = new Semaphore(capacity);
        items = (E[]) new Object[capacity];
    }
    public boolean isEmpty() {
        return availableItems.availablePermits() == 0;
    }
    public boolean isFull() {
        return availableSpaces.availablePermits() == 0;
    }

    public void put(E x) throws InterruptedException {
        availableSpaces.acquire();
        doInsert(x);
        availableItems.release();
    }
    public E take() throws InterruptedException {
        availableItems.acquire();
        E item = doExtract();
        availableSpaces.release();
        return item;
    }

    private synchronized void doInsert(E x) {
        int i = putPosition;
        items[i] = x;
        putPosition = (++i == items.length)? 0 : i;
    }
    private synchronized E doExtract() {
        int i = takePosition;
        E x = items[i];
        items[i] = null;
```

```
        takePosition = (++i == items.length)? 0 : i;
        return x;
    }
}
```

BoundedBuffer 实现了一个固定长度的队列，其中定义了可阻塞的 put 和 take 方法，并通过两个计数信号量进行控制。信号量 availableItems 表示可以从缓存中删除的元素个数，它的初始值为零（因为缓存的初始状态为空）。同样，信号量 availableSpaces 表示可以插入到缓存的元素个数，它的初始值等于缓存的大小。

take 操作首先请求从 availableItems 中获得一个许可 (Permit)。如果缓存不为空，那么这个请求会立即成功，否则请求将被阻塞直到缓存不为空。在获得了一个许可后，take 方法将删除缓存中的下一个元素，并返回一个许可到 availableSpaces 信号量[⊖]。put 方法的执行顺序刚好相反，因此无论是从 put 方法还是从 take 方法中退出，这两个信号量计数值的加和都会等于缓存的大小。（在实际情况中，如果需要一个有界缓存，应该直接使用 ArrayBlockingQueue 或 LinkedBlockingQueue，而不是自己编写，但这里用于说明如何对添加和删除等方法进行控制的技术，在其他数据结构中同样可以使用。）

12.1.1 基本的单元测试

BoundedBuffer 的最基本单元测试类似于在串行上下文中执行的测试。首先创建一个有界缓存，然后调用它的各个方法，并验证它的后验条件和不变性条件。我们很快会想到一些不变性条件：新建立的缓存应该是空的，而不是满的。另一个略显复杂的安全测试是，将 N 个元素插入到容量为 N 的缓存中（这个过程应该可以成功，并且不会阻塞），然后测试缓存是否已经填满（不为空）。程序清单 12-2 给出了这些属性的 JUnit 测试方法。

程序清单 12-2 BoundedBuffer 的基本单元测试

```
class BoundedBufferTest extends TestCase {
    void testIsEmptyWhenConstructed() {
        BoundedBuffer<Integer> bb = new BoundedBuffer<Integer>(10);
        assertTrue(bb.isEmpty());
        assertFalse(bb.isFull());
    }

    void testIsFullAfterPuts() throws InterruptedException {
        BoundedBuffer<Integer> bb = new BoundedBuffer<Integer>(10);
        for (int i = 0; i < 10; i++)
            bb.put(i);
        assertTrue(bb.isFull());
        assertFalse(bb.isEmpty());
    }
}
```

⊖ 在计数信号量中，通常不会显式地用一个所有者线程来表示许可，或者将其与所有者线程关联起来。release 方法将创建一个许可，而 acquire 方法将消耗一个许可。

这些简单的测试方法都是串行的。在测试集中包含一组串行测试通常是有帮助的，因为它们有助于在开始分析数据竞争之前就找出与并发性无关的问题。

12.1.2 对阻塞操作的测试

在测试并发的基本属性时，需要引入多个线程。大多数测试框架并不能很好地支持并发性测试：它们很少会包含相应的工具来创建线程或监视线程，以确保它们不会意外结束。如果在某个测试用例创建的辅助线程中发现了一个错误，那么框架通常无法得知与这个线程相关的是哪一个测试，所以需要通过一些工作将成功或失败信息传递回主测试线程，从而才能将相应的信息报告出来。

在 java.util.concurrent 的一致性测试中，一定要将各种故障与特定的测试明确地关联起来。因此 JSR 166 专家组创建了一个基类[⊖]，其中定义了一些方法可以在 tearDown 期间传递和报告失败信息，并遵循一个约定：每个测试必须等待它所创建的全部线程结束以后才能完成。你不需要考虑这么深入，关键的需求在于，能否通过这些测试，以及是否在某个地方报告了失败信息以便用于诊断问题。

如果某方法需要在某些特定条件下阻塞，那么当测试这种行为时，只有当线程不再继续执行时，测试才是成功的。要测试一个方法的阻塞行为，类似于测试一个抛出异常的方法：如果这个方法可以正常返回，那么就意味着测试失败。

在测试方法的阻塞行为时，将引入额外的复杂性：当方法被成功地阻塞后，还必须使方法解除阻塞。实现这个功能的一种简单方式就是使用中断——在一个单独的线程中启动一个阻塞操作，等到线程阻塞后再中断它，然后宣告阻塞操作成功。当然，这要求阻塞方法通过提前返回或者抛出 InterruptedException 来响应中断。

“等待并直到线程阻塞后”这句话说起来简单，做起来难。实际上，你必须估计执行这些指令可能需要多长的时间，并且等待的时间会更长。如果估计的时间不准确（在这种情况下，你会看到伪测试失败），那么应该增大这个值。

程序清单 12-3 给出了一种测试阻塞操作的方法。这种方法会创建一个“获取”线程，该线程将尝试从空缓存中获取一个元素。如果 take 方法成功，那么表示测试失败。执行测试的线程启动“获取”线程，等待一段时间，然后中断该线程。如果“获取”线程正确地在 take 方法中阻塞，那么将抛出 InterruptedException，而捕获到这个异常的 catch 块将把这个异常视为测试成功，并让线程退出。然后，主测试线程会尝试与“获取”线程合并，通过调用 Thread.isAlive 来验证 join 方法是否成功返回，如果“获取”线程可以响应中断，那么 join 能很快地完成。

程序清单 12-3 测试阻塞行为以及对中断的响应性

```
void testTakeBlocksWhenEmpty() {
    final BoundedBuffer<Integer> bb = new BoundedBuffer<Integer>(10);
    Thread taker = new Thread() {
        public void run() {
```

⊖ http://gee.cs.oswego.edu/cgi-bin/viewcvs.cgi/jsr166/src/test/tck/JSR166TestCase.java。

```
                try {
                    int unused = bb.take();
                    fail();  // 如果执行到这里，那么表示出现了一个错误
                } catch (InterruptedException success) { }
            }};
        try {
            taker.start();
            Thread.sleep(LOCKUP_DETECT_TIMEOUT);
            taker.interrupt();
            taker.join(LOCKUP_DETECT_TIMEOUT);
            assertFalse(taker.isAlive());
        } catch (Exception unexpected) {
            fail();
        }
    }
```

如果 take 操作由于某种意料之外的原因停滞了，那么支持限时的 join 方法能确保测试最终完成。这个测试方法测试了 take 的多种属性——不仅能阻塞，而且在中断后还能抛出 InterruptedException。在这种情况下，最好是对 Thread 进行子类化而不是使用线程池中的 Runnable，即通过 join 来正确地结束测试。当主线程将一个元素放入队列后，“获取”线程应该解除阻塞状态，要测试这种行为，可以使用相同的方法。

开发人员会尝试使用 Thread.getState 来验证线程能否在一个条件等待上阻塞，但这种方法并不可靠。被阻塞线程并不需要进入 WAITING 或 TIMED_WAITING 等状态，因此 JVM 可以选择通过自旋等待来实现阻塞。类似地，由于在 Object.wait 或 Condition.await 等方法上存在伪唤醒（Spurious Wakeup，请参见第 14 章），因此，即使一个线程等待的条件尚未成真，也可能从 WAITING 或 TIMED_WAITING 等状态临时性地转换到 RUNNABLE 状态。即使忽略这些不同实现之间的差异，目标线程在进入阻塞状态时也会消耗一定的时间。Thread.getState 的返回结果不能用于并发控制，它将限制测试的有效性——其主要作用还是作为调试信息的来源。

12.1.3 安全性测试

程序清单 12-2 和程序清单 12-3 的测试用例测试了有界缓存的一些重要属性，但它们却无法发现由于数据竞争而引发的错误。要想测试一个并发类在不可预测的并发访问情况下能否正确执行，需要创建多个线程来分别执行 put 和 take 操作，并在执行一段时间后判断在测试中是否会出现问题。

如果要构造一些测试来发现并发类中的安全性错误，那么这实际上是一个“先有蛋还是先有鸡”的问题：测试程序自身就是并发程序。要开发一个良好的并发测试程序，或许比开发这些程序要测试的类更加困难。

> 在构建对并发类的安全性测试中，需要解决的关键问题在于，要找出那些容易检查的属性，这些属性在发生错误的情况下极有可能失败，同时又不会使得错误检查代码人为地限制并发性。理想情况是，在测试属性中不需要任何同步机制。

要测试在生产者－消费者模式中使用的类，一种有效的方法就是检查被放入队列中和从队列中取出的各个元素。这种方法的一种简单实现是，当元素被插入到队列时，同时将其插入到一个“影子”列表，当从队列中删除该元素时，同时也从“影子”列表中删除，然后在测试程序运行完以后判断“影子”列表是否为空。然而，这种方法可能会干扰测试线程的调度，因为在修改“影子”列表时需要同步，并可能会阻塞。

一种更好的方法是，通过一个对顺序敏感的校验和计算函数来计算所有入列元素以及出列元素的校验和，并进行比较。如果二者相等，那么测试就是成功的。如果只有一个生产者将元素放入缓存，同时也只有一个消费者从中取出元素，那么这种方法能发挥最大的作用，因为它不仅能测试出是否取出了正确的元素，而且还能测试出元素被取出的顺序是否正确。

如果要将这种方法扩展到多生产者－多消费者的情况，就需要一个对元素入列/出列顺序不敏感的校验和函数，从而在测试程序运行完以后，可以将多个校验和以不同的顺序组合起来。如果不是这样，多个线程就需要访问同一个共享的校验和变量，因此就需要同步，这将成为一个并发的瓶颈或者破坏测试的执行时序。（任何具备可交换性的操作，例如加法或 XOR，都符合这些需求。）

要确保测试程序能正确地测试所有要点，就一定不能让编译器可以预先猜测到校验和的值。使用连续的整数作为测试数据并不是一种好办法，因为得到的结果总是相同的，而一个智能的编译器通常可以预先计算出这个结果。

要避免这个问题，应该采用随机方式生成的测试数据，但如果选择了一种不合适的随机数生成器（RNG，Random Number Generator），那么会对许多其他的测试造成影响。由于大多数随机数生成器类都是线程安全的，并且会带来额外的同步开销[⊖]，因此在随机数生成过程中，可能会在这些类与执行时序之间产生耦合关系。如果每个线程都拥有各自的 RNG，那么这些 RNG 就可以不是线程安全的。

与其使用一个通用的 RNG，还不如使用一些简单的伪随机函数。你并不需要某种高质量的随机性，而只需要确保在不同的测试运行中都有不同的数字。程序清单 12-4 的 xorShift 函数 (Marsaglia, 2003) 是最符合这个需求的随机数函数之一。该函数基于 hashCode 和 nanoTime 来生成随机数，所得的结果既是不可预测的，而且基本上每次运行都不同。

程序清单 12-4 适合在测试中使用的随机数生成器

```
static int xorShift(int y) {
    y ^= (y << 6);
    y ^= (y >>> 21);
    y ^= (y << 7);
    return y;
}
```

在程序清单 12-5 和程序清单 12-6 的 PutTakeTest 中启动了 *N* 个生产者线程来生成元素并把它们插入到队列，同时还启动了 *N* 个消费者线程从队列中取出元素。当元素进出队列时，每个

⊖ 有很多基准测试都是衡量 RNG 造成的并行瓶颈有多大，而许多开发人员或用户并不知道这些基准的存在。

线程都会更新对这些元素计算得到的校验和，每个线程都拥有一个校验和，并在测试结束后将它们合并起来，从而在测试缓存时就不会引入过多的同步或竞争。

程序清单 12-5 测试 BoundedBuffer 的生产者－消费者程序

```
public class PutTakeTest {
    private static final ExecutorService pool
            = Executors.newCachedThreadPool();
    private final AtomicInteger putSum = new AtomicInteger(0);
    private final AtomicInteger takeSum = new AtomicInteger(0);
    private final CyclicBarrier barrier;
    private final BoundedBuffer<Integer> bb;
    private final int nTrials, nPairs;

    public static void main(String[] args) {
    new PutTakeTest(10, 10, 100000).test(); // 示例参数
        pool.shutdown();
    }

    PutTakeTest(int capacity, int npairs, int ntrials) {
        this.bb = new BoundedBuffer<Integer>(capacity);
        this.nTrials = ntrials;
        this.nPairs = npairs;
        this.barrier = new CyclicBarrier(npairs*  2 + 1);
    }

    void test() {
        try {
            for (int i = 0; i <  nPairs; i++) {
                pool.execute(new Producer());
                pool.execute(new Consumer());
            }
            barrier.await(); // 等待所有的线程就绪
            barrier.await(); // 等待所有的线程执行完成
            assertEquals(putSum.get(), takeSum.get());
        } catch (Exception e) {
            throw new RuntimeException(e);
        }
    }

class Producer implements Runnable {/* 程序清单 12-6*/}

class Consumer implements Runnable {/* 程序清单 12-6*/}
}
```

程序清单 12-6 在 PutTakeTest 中使用的 Producer 和 Consumer 等类

```
/*PutTakeTest 的内部类（程序清单 12-5）*/
class Producer implements Runnable {
    public void run() {
        try {
            int seed = (this.hashCode() ^ (int)System.nanoTime());
            int sum = 0;
```

```
            barrier.await();
            for (int i = nTrials; i > 0; --i) {
                bb.put(seed);
                sum += seed;
                seed = xorShift(seed);
            }
            putSum.getAndAdd(sum);
            barrier.await();
        } catch (Exception e) {
            throw new RuntimeException(e);
        }
    }
}

class Consumer implements Runnable {
    public void run() {
        try {
            barrier.await();
            int sum = 0;
            for (int i = nTrials; i > 0; --i) {
                sum += bb.take();
            }
            takeSum.getAndAdd(sum);
            barrier.await();
        } catch (Exception e) {
            throw new RuntimeException(e);
        }
    }
}
```

根据系统平台的不同，创建线程与启动线程等操作可能需要较大开销。如果线程的执行时间很短，并且在循环中启动了大量的这种线程，那么最坏的情况就是，这些线程将会串行执行而不是并发执行。即使在一些不太糟糕的情况下，第一个线程仍然比其他线程具有“领先优势”。因此这可能无法获得预想中的交替执行：第一个线程先运行一段时间，然后前两个线程会并发地运行一段时间，只有到了最后，所有线程才会一起并发执行。（在线程结束运行时存在同样的问题：第一个运行的线程将提前完成。）

在 5.5.1 节中介绍了一项可以缓解这个问题的技术，即两个 CountDownLatch，其中一个作为开始阀门，而另一个作为结束阀门。使用 CyclicBarrier 也可以获得同样的效果：在初始化 CyclicBarrier 时将计数值指定为工作者线程的数量再加 1，并在运行开始和结束时，使工作者线程和测试线程都在这个栅栏处等待。这能确保所有线程在开始执行任何工作之前，都首先执行到同一个位置。PutTakeTest 使用这项技术来协调工作者线程的启动和停止，从而产生更多的并发交替操作。我们仍然无法确保调度器不会采用串行方式来执行每个线程，但只要这些线程的执行时间足够长，就能降低调度机制对结果的不利影响。

PutTakeTest 使用了一个确定性的结束条件，从而在判断测试何时完成时就不需要在线程之间执行额外的协调。test 方法将启动相同数量的生产者线程和消费者线程，它们将分别插入 (put) 和取出（Take）相同数量的元素，因此添加与删除的总数相同。

像 PutTakeTest 这种测试能很好地发现安全性问题。例如，在实现由信号量控制的缓存时，一个常见的错误就是在执行插入和取出的代码中忘记实现互斥行为（可以使用 synchronized 或

ReentrantLock）。如果在 PutTakeTesc 使用的 BoundedBuffer 中忘记将 doInsert 和 doExtract 声明为 synchronized，那么在运行 PutTakeTest 时会立即失败。通过多个线程来运行 PutTakeTest，并且使这些线程在不同系统上的不同容量的缓存上迭代数百万次，使我们能进一步确定在 put 和 take 方法中不存在数据破坏问题。

这些测试应该放在多处理器的系统上运行，从而进一步测试更多形式的交替运行。然而，CPU 的数量越多并不一定会使测试越高效。要最大程度地检测出一些对执行时序敏感的数据竞争，那么测试中的线程数量应该多于 CPU 数量，这样在任意时刻都会有一些线程在运行，而另一些被交换出去，从而可以检查线程间交替行为的可预测性。

在一些测试中通常要求执行完一定数量的操作后才能停止运行，如果在测试代码中出现了一个错误并抛出了一个异常，那么这个测试将永远不会结束。最常见的解决方法是：让测试框架放弃那些没有在规定时间内完成的测试，具体要等待多长的时间，则要凭经验来确定，并且要对故障进行分析以确保所出现的问题并不是由于没有等待足够长的时间而造成的。（这个问题并不仅限于对并发类的测试，在串行测试中也必须区分长时间的运行和无限循环。）

12.1.4　资源管理的测试

到目前为止，所有的测试都侧重于类与它的设计规范的一致程度——在类中应该实现规范中定义的功能。测试的另一个方面就是要判断类中是否没有做它不应该做的事情，例如资源泄漏。对于任何持有或管理其他对象的对象，都应该在不需要这些对象时销毁对它们的引用。这种存储资源泄漏不仅会妨碍垃圾回收器回收内存（或者线程、文件句柄、套接字、数据库连接或其他有限资源），而且还会导致资源耗尽以及应用程序失败。

对于像 BoundedBuffer 这样的类来说，资源管理的问题尤为重要。之所以要限制缓存的大小，其原因就是要防止由于资源耗尽而导致应用程序发生故障，例如生产者的速度远远高于消费者的处理速度。通过对缓存进行限制，将使得生产力过剩的生产者被阻塞，因而它们就不会继续创建更多的工作来消耗越来越多的内存以及其他资源。

通过一些测量应用程序中内存使用情况的堆检查工具，可以很容易地测试出对内存的不合理占用，许多商业和开源的堆分析工具中都支持这种功能。在程序清单 12-7 的 testLeak 方法中包含了一些堆分析工具用于抓取堆的快照，这将强制执行一次垃圾回收[⊖]，然后记录堆大小和内存用量的信息。

程序清单 12-7　测试资源泄漏

```
class Big { double[] data = new double[100000]; }

void testLeak() throws InterruptedException {
```

⊖ 从技术上来看，我们无法强制执行一次垃圾回收，System.gc 只是建议 JVM 在合适的时刻执行垃圾回收。通过使用 -XX:+DisableExplicitGC，可以告诉 HotSpot 忽略 System.gc 调用。

```
    BoundedBuffer<Big> bb = new BoundedBuffer<Big>(CAPACITY);
int heapSize1 =  /* 生成堆的快照 */ ;
    for (int i = 0; i < CAPACITY; i++)
        bb.put(new Big());
    for (int i = 0; i < CAPACITY; i++)
        bb.take();
int heapSize2 =  /* 生成堆的快照 */ ;
    assertTrue(Math.abs(heapSize1-heapSize2) < THRESHOLD);
}
```

testLeak 方法将多个大型对象插入到一个有界缓存中，然后再将它们移除。第 2 个堆快照中的内存用量应该与第 1 个堆快照中的内存用量基本相同。然而，doExtract 如果忘记将返回元素的引用置为空 (items[i] =null)，那么在两次快照中报告的内存用量将明显不同。（这是为数不多几种需要显式地将变量置空的情况之一。在大多数情况下，这种做法不仅不会带来帮助，甚至还会带来负面作用 [EJ Item 5]。）

12.1.5 使用回调

在构造测试案例时，对客户提供的代码进行回调是非常有帮助的。回调函数的执行通常是在对象生命周期的一些已知位置上，并且在这些位置上非常适合判断不变性条件是否被破坏。例如，在 ThreadPoolExecutor 中将调用任务的 Runnable 和 ThreadFactory。

在测试线程池时，需要测试执行策略的多个方面：在需要更多的线程时创建新线程，在不需要时不创建，以及当需要回收空闲线程时执行回收操作等。要构造一个全面的测试方案是很困难的，但其中许多方面的测试都可以单独进行。

通过使用自定义的线程工厂，可以对线程的创建过程进行控制。在程序清单 12-8 的 TestingThreadFactory 中将记录已创建线程的数量。这样，在测试过程中，测试方案可以验证已创建线程的数量。我们还可以对 TestingThreadFactory 进行扩展，使其返回一个自定义的 Thread，并且该对象可以记录自己在何时结束，从而在测试方案中验证线程在被回收时是否与执行策略一致。

程序清单 12-8 测试 THReadPoolExecutor 的线程工厂类

```
class TestingThreadFactory implements ThreadFactory {
    public final AtomicInteger numCreated = new AtomicInteger();
    private final ThreadFactory factory
            = Executors.defaultThreadFactory();

    public Thread newThread(Runnable r) {
        numCreated.incrementAndGet();
        return factory.newThread(r);
    }
}
```

如果线程池的基本大小小于最大大小，那么线程池会根据执行需求相应增长。当把一些运行时间较长的任务提交给线程池时，线程池中的任务数量在长时间内都不会变化，这就可以进行一些判断，例如测试线程池是否能按照预期的方式扩展，如程序清单 12-9 所示。

程序清单 12-9 验证线程池扩展能力的测试方法

```
public void testPoolExpansion() throws InterruptedException {
    int MAX_SIZE = 10;
    ExecutorService exec = Executors.newFixedThreadPool(MAX_SIZE);

    for (int i = 0; i < 10*  MAX_SIZE; i++)
        exec.execute(new Runnable() {
            public void run() {
                try {
                    Thread.sleep(Long.MAX_VALUE);
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }
        });
    for (int i = 0;
         i < 20 && threadFactory.numCreated.get() < MAX_SIZE;
         i++)
        Thread.sleep(100);
    assertEquals(threadFactory.numCreated.get(), MAX_SIZE);
    exec.shutdownNow();
}
```

12.1.6 产生更多的交替操作

由于并发代码中的大多数错误都是一些低概率事件，因此在测试并发错误时需要反复地执行许多次，但有些方法可以提高发现这些错误的概率。在前面提到过，在多处理器系统上，如果处理器的数量少于活动线程的数量，那么与单处理器系统或者包含多个处理器的系统相比，将能产生更多的交替行为。同样，如果在不同的处理器数量、操作系统以及处理器架构的系统上进行测试，就可以发现那些在特定运行环境中才会出现的问题。

有一种有用的方法可以提高交替操作的数量，以便能更有效地搜索程序的状态空间：在访问共享状态的操作中，使用 Thread.yield 将产生更多的上下文切换。（这项技术的有效性与具体的平台相关，因为 JVM 可以将 Thread.yield 作为一个空操作（no-op）[JLS 17.9]。如果使用一个睡眠时间较短的 sleep，那么虽然更慢些，但却更可靠。）程序清单 12-10 中的方法在两个账户之间执行转账操作，在两次更新操作之间，像“所有账户的总和应等于零”这样的一些不变性条件可能会被破坏。当代码在访问状态时没有使用足够的同步，将存在一些对执行时序敏感的错误，通过在某个操作的执行过程中调用 yield 方法，可以将这些错误暴露出来。这种方法需要在测试中添加一些调用并且在正式产品中删除这些调用，这将给开发人员带来不便，通过使用面向方面编程（Aspect-Oriented Programming，AOP）的工具，可以降低这种不便性。

程序清单 12-10 使用 Thread.yield 来产生更多的交替操作

```
public synchronized void transferCredits(Account from,
                                         Account to,
                                         int amount) {
```

```
        from.setBalance(from.getBalance() - amount);
        if (random.nextInt(1000) > THRESHOLD)
            Thread.yield();
        to.setBalance(to.getBalance() + amount);
    }
```

12.2 性能测试

性能测试通常是功能测试的延伸。事实上，在性能测试中应该包含一些基本的功能测试，从而确保不会对错误的代码进行性能测试。

虽然在性能测试与功能测试之间肯定会存在重叠之处，但它们的目标是不同的。性能测试将衡量典型测试用例中的端到端性能。通常，要获得一组合理的使用场景并不容易，理想情况下，在测试中应该反映出被测试对象在应用程序中的实际用法。

在某些情况下，也存在某种显而易见的测试场景。在生产者－消费者设计中通常都会用到有界缓存，因此显然需要测试生产者在向消费者提供数据时的吞吐量。对 PutTakeTest 进行扩展，使其成为针对该使用场景的性能测试。

性能测试的第二个目标是根据经验值来调整各种不同的限值，例如线程数量、缓存容量等。这些限值可能依赖于具体平台的特性（例如，处理器的类型、处理器的步进级别（Stepping Level）、CPU 的数量或内存大小等），因此需要动态地进行配置，而我们通常需要合理地选择这些值，从而使程序能够在更多的系统上良好地运行。

12.2.1 在 PutTakeTest 中增加计时功能

之前对 PutTakeTest 的主要扩展就是测量运行一次所需的时间。现在，我们不测量单个操作的时间，而是实现一种更精确的测量方式：记录整个运行过程的时间，然后除以总操作的数量，从而得到每次操作的运行时间。之前使用了 CyclicBarrier 来启动和结束工作者线程，因此可以对其进行扩展：使用一个栅栏动作来测量启动和结束时间，如程序清单 12-11 所示。

程序清单 12-11 基于栅栏的定时器

```
this.timer = new BarrierTimer();
this.barrier = new CyclicBarrier(npairs * 2 + 1, timer);
public class BarrierTimer implements Runnable {
    private boolean started;
    private long startTime, endTime;

    public synchronized void run() {
        long t = System.nanoTime();
        if (!started) {
            started = true;
            startTime = t;
        } else
            endTime = t;
    }
    public synchronized void clear() {
        started = false;
```

```
    }
    public synchronized long getTime() {
        return endTime - startTime;
    }
}
```

我们可以将栅栏的初始化过程修改为使用这种栅栏动作，即使用能接受栅栏动作的CyclicBarrier 构造函数。

在修改后的 test 方法中使用了基于栅栏的计时器，如程序清单 12-12 所示。

程序清单 12-12 采用基于栅栏的定时器进行测试

```
public void test() {
    try {
        timer.clear();
        for (int i = 0; i < nPairs; i++) {
            pool.execute(new Producer());
            pool.execute(new Consumer());
        }
        barrier.await();
        barrier.await();
        long nsPerItem = timer.getTime() / (nPairs*  (long)nTrials);
        System.out.print("Throughput: " + nsPerItem + " ns/item");
        assertEquals(putSum.get(), takeSum.get());
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}
```

我们可以从 TimedPutTakeTest 的运行中学到一些东西。第一，生产者－消费者模式在不同参数组合下的吞吐率。第二，有界缓存在不同线程数量下的可伸缩性。第三，如何选择缓存的大小。要回答这些问题，需要对不同的参数组合进行测试，因此我们需要一个主测试驱动程序，如程序清单 12-13 所示。

程序清单 12-13 使用 TimedPutTakeTest 的程序

```
public static void main(String[] args) throws Exception {
int tpt = 100000;  // 每个线程中的测试次数
    for (int cap = 1; cap <= 1000; cap*= 10) {
        System.out.println("Capacity: " + cap);
        for (int pairs = 1; pairs <= 128; pairs*= 2) {
            TimedPutTakeTest t =
                new TimedPutTakeTest(cap, pairs, tpt);
            System.out.print("Pairs: " + pairs + "\t");
            t.test();
            System.out.print("\t");
            Thread.sleep(1000);
            t.test();
            System.out.println();
            Thread.sleep(1000);
```

```
        }
    }
    pool.shutdown();
}
```

图 12-1 给出了在 4 路机器上的一些测试结果，缓存的容量分别为 1、10、100 和 1000。我们可以看到，当缓存大小为 1 时，吞吐率是非常糟糕的，这是因为每个线程在阻塞并且等待另一个线程之前，所取得的进展是非常有限的。当把缓存大小提高到 10 时，吞吐率得到了极大提高：但在超过 10 之后，所得到的收益又开始降低。

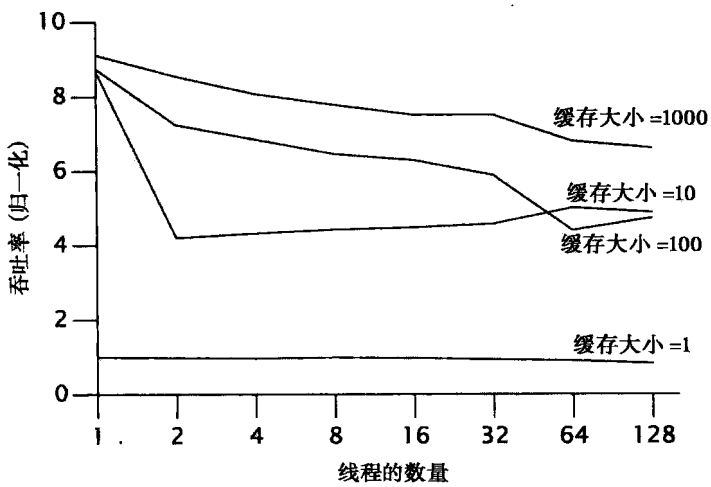

图 12-1　在不同缓存大小下的 TimedPutTakeTest

初看起来会感到有些困惑：当增加更多的线程时，性能却略有下降。其中的原因很难从数据中看出来，但可以在运行测试时使用 CPU 的性能工具（例如 perfbar）：虽然有许多的线程，但却没有足够多的计算量，并且大部分时间都消耗在线程的阻塞和解除阻塞等操作上。线程有足够多的 CPU 空闲时钟周期来做相同的事情，因此不会过多地降低性能。

然而，要谨慎对待从上面数据中得出的结论，即在使用有界缓存的生产者－消费者程序中总是可以添加更多的线程。这个测试在模拟应用程序时忽略了许多实际的因素，例如生产者几乎不需要任何工作就可以生成一个元素并将它放入队列，同时消费者也无须太多工作就能获取一个元素。在真实的生产者－消费者应用程序中，如果工作者线程要通过执行一些复杂的操作来生产和获取各个元素条目（通常就是这种情况），那么之前那种 CPU 空闲状态将消失，并且由于线程过多而导致的影响将变得非常明显。这个测试的主要目的是，测量生产者和消费者在通过有界缓存传递数据时，哪些约束条件将对整体吞吐量产生影响。

12.2.2　多种算法的比较

虽然 BoundedBuffer 是一种非常合理的实现，并且它的执行性能也不错，但还是没有 ArrayBlockingQueue 或 LinkedBlockingQueue 那样好（这也解释了为什么这种缓存算法没有被

选入类库中）。java.util.concurrent 中的算法已经通过类似的测试进行了调优，其性能也已经达到我们已知的最佳状态。此外，这些算法还能提供更多的功能[⊖]。BoundedBuffer 运行效率不高的主要原因是：在 put 和 take 方法中都含有多个可能发生竞争的操作，例如，获取一个信号量，获取一个锁，以及释放信号量等。在其他实现方法中，可能发生竞争的位置将少很多。

图 12-2 给出了一个在双核超线程机器上对这三个类的吞吐量测试结果，在测试中使用了一个包含 256 个元素的缓存，以及相应版本的 TimedPutTakeTest。测试结果表明，LinkedBlockingQueue 的可伸缩性要高于 ArrayBlockingQueue。初看起来，这个结果有些奇怪：链表队列在每次插入元素时，都必须分配一个链表节点对象，这似乎比基于数组的队列执行了更多的工作。然而，虽然它拥有更好的内存分配与 GC 等开销，但与基于数组的队列相比，链表队列的 put 和 take 等方法支持并发性更高的访问，因为一些优化后的链接队列算法能将队列头节点的更新操作与尾节点的更新操作分离开来。由于内存分配操作通常是线程本地的，因此如果算法能通过多执行一些内存分配操作来降低竞争程度，那么这种算法通常具有更高的可伸缩性。（这种情况再次证明了，基于传统性能调优的直觉与提升可伸缩性的实际需求是背道而驰的。）

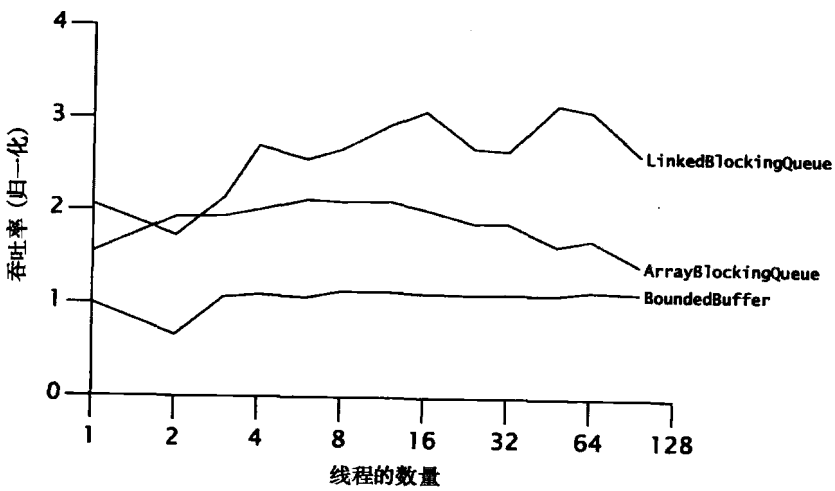

图 12-2　比较不同的阻塞队列实现

12.2.3　响应性衡量

到目前为止，我们的重点是吞吐量的测量，这通常是并发程序最重要的性能指标。但有时候，我们还需要知道某个动作经过多长时间才能执行完成，这时就要测量服务时间的变化情况。而且，如果能获得更小的服务时间变动性，那么更长的平均服务时间是有意义的，“可预测性”同样是一个非常有价值的性能特征。通过测量变动性，使我们能回答一些关于服务质量的问题，例如“操作在 100 毫秒内成功执行的百分比是多少？”

⊖ 如果你是一个并发专家，并且放弃一些已提供的功能，那么就可能超过它们的性能。

通过表示任务完成时间的直方图，最能看出服务时间的变动。服务时间变动的测量比平均值的测量要略困难一些——除了总共完成时间外，还要记录每个任务的完成时间。因为计时器的粒度通常是测量任务时间的一个主要因素（任务的执行时间可能小于或接近于最小“定时器计时单位”，这将影响测量结果的精确性），为了避免测量过程中的人为影响，我们可以测量一组 put 和 take 方法的运行时间。

图 12-3 给出了在不同 TimedPutTakeTest 中每个任务的完成时间，其中使用了一个大小为 1000 的缓存，有 256 个并发任务，并且每个任务都将使用非公平的信号量（隐蔽栅栏，Shaded Bars）和公平的信号量（开放栅栏，open bars）来迭代这 1000 个元素。(13.3 节将介绍锁和信号量的公平排队与非公平排队。）非公平信号量完成时间的变动范围为 104 到 8714 毫秒，相差超过 80 倍。通过在同步控制中实现更高的公平性，可以缩小这种变动范围，通过在 BoundedBuffer 中将信号量初始化为公平模式，可以很容易实现这个功能。如图 12-3 所示，这种方法能成功地降低变动性（现在的变动范围为 38194 到 38207 毫秒），然而，该方法会极大地降低吞吐量。（如果在一个运行时间较长的测试中执行更多种任务，那么吞吐量的下降程度可能更大。）

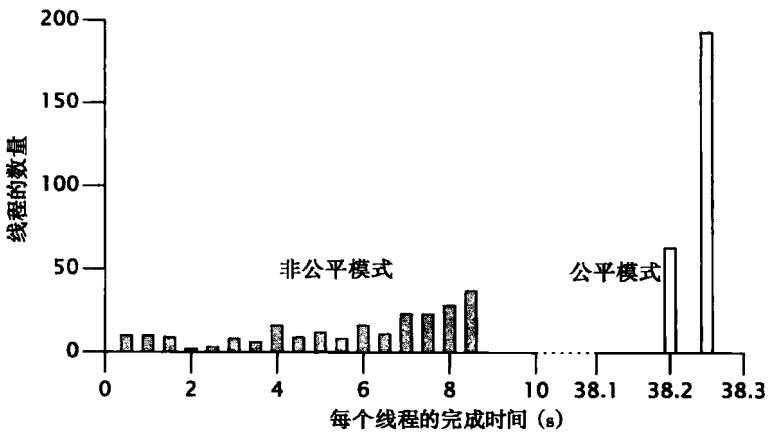

图 12-3　TimedPutTakeTest 在使用默认（非公平）信号量与公平信号量下的完成时间直方图

前面曾讨论过，如果缓存过小，那么将导致非常多的上下文切换次数，这即使在非公平模式中也会导致很低的吞吐量，因此在几乎每个操作中都会执行上下文切换。为了说明公平性开销主要是由于线程阻塞而造成的，我们可以将缓存大小设置为 1，然后重新运行这个测试，从而可以看出此时非公平信号量与公平信号量的执行性能基本相当。如图 12-4 所示，这种情况下公平性并不会使平均完成时间变长，或者使变动性变小。

因此，除非线程由于密集的同步需求而被持续地阻塞，否则非公平的信号量通常能实现更好的吞吐量，而公平的信号量则实现更低的变动性。因为这些结果之间的差异非常大，所以 Semaphore 要求客户选择针对哪一个特性进行优化。

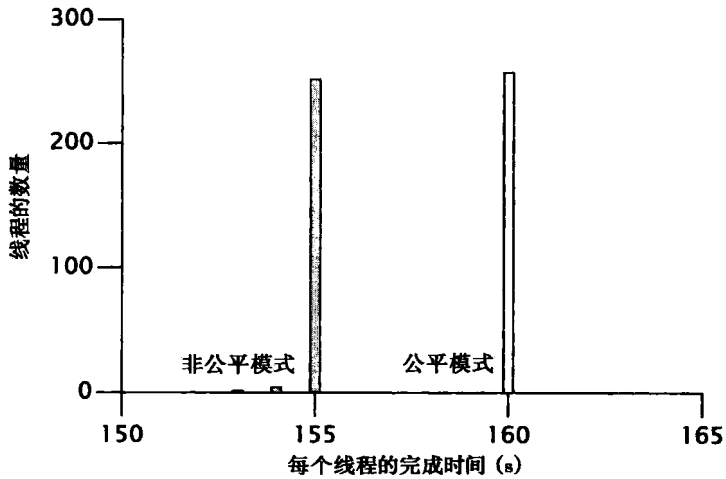

图 12-4 TimedPutTakeTest 在使用单元素缓存时的完成时间直方图

12.3 避免性能测试的陷阱

理论上，开发性能测试程序是很容易的——找出一个典型的使用场景，编写一段程序多次执行这种使用场景，并统计程序的运行时间。但在实际情况中，你必须提防多种编码陷阱，它们会使性能测试变得毫无意义。

12.3.1 垃圾回收

垃圾回收的执行时序是无法预测的，因此在执行测试时，垃圾回收器可能在任何时刻运行。如果测试程序执行了 N 次迭代都没有触发垃圾回收操作，但在第 N+1 次迭代时触发了垃圾回收操作，那么即使运行次数相差不大，仍可能在最终测试的每次迭代时间上带来很大的（但却虚假的）影响。

有两种策略可以防止垃圾回收操作对测试结果产生偏差。第一种策略是，确保垃圾回收操作在测试运行的整个期间都不会执行（可以在调用 JVM 时指定 -verbose：gc 来判断是否执行了垃圾回收操作）。第二种策略是，确保垃圾回收操作在测试期间执行多次，这样测试程序就能充分反映出运行期间的内存分配与垃圾回收等开销。通常第二策略更好，它要求更长的测试时间，并且更有可能反映实际环境下的性能。

在大多数采用生产者－消费者设计的应用程序中，都会执行一定数量的内存分配与垃圾回收等操作——生产者分配新对象，然后被消费者使用并丢弃。如果将有界缓存测试运行足够长的时间，那么将引发多次垃圾回收，从而得到更精确的结果。

12.3.2 动态编译

与静态编译语言（例如，C 或 C++）相比，编写动态编译语言（例如 Java）的性能基准测试要困难得多。在 HotSpot JVM（以及其他现代的 JVM）中将字节码的解释与动态编译结合起

来使用。当某个类第一次被加载时，JVM 会通过解译字节码的方式来执行它。在某个时刻，如果一个方法运行的次数足够多，那么动态编译器会将它编译为机器代码，当编译完成后，代码的执行方式将从解释执行变成直接执行。

这种编译的执行时机是无法预测的。只有在所有代码都编译完成以后，才应该统计测试的运行时间。测量采用解释执行的代码速度是没有意义的，因为大多数程序在运行足够长的时间后，所有频繁执行的代码路径都会被编译。如果编译器可以在测试期间运行，那么将在两个方面对测试结果带来偏差：在编译过程中将消耗 CPU 资源，并且，如果在测量的代码中既包含解释执行的代码，又包含编译执行的代码，那么通过测试这种混合代码得到的性能指标没有太大意义。图 12-5 给出了动态编译在测试结果上带来的偏差。这 3 条时间线表示执行了相同次数的迭代：时间线 A 表示所有代码都采用解释执行，时间线 B 表示在运行过程中间开始转向编译执行，而时间线 C 表示从较早时刻就开始采用编译执行。编译执行的开始时刻会对每次操作的运行时间产生极大的影响。[⊖]

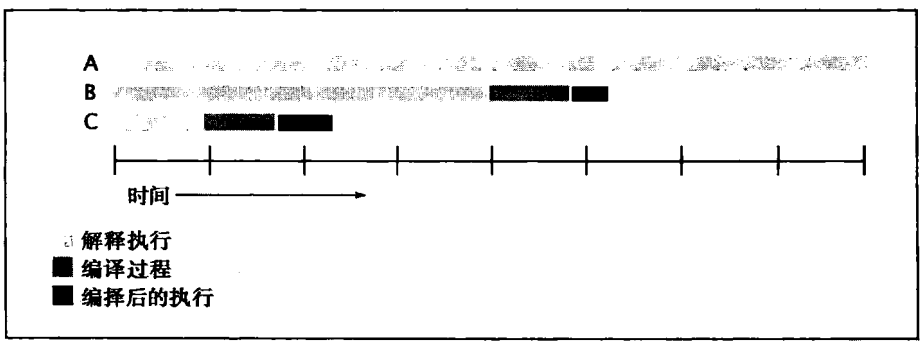

图 12-5　动态编译对测试结果带来的偏差

基于各种原因，代码还可能被反编译（退回到解释执行）以及重新编译，例如加载了一个会使编译假设无效的类，或者在收集了足够的分析信息后，决定采用不同的优化措施来重编译某条代码路径。

有一种方式可以防止动态编译对测试结果产生偏差，就是使程序运行足够长的时间（至少数分钟），这样编译过程以及解释执行都只是总运行时间的很小一部分。另一种方法是使代码预先运行一段时间并且不测试这段时间内的代码性能，这样在开始计时前代码就已经被完全编译了。在 HotSpot 中，如果在运行程序时使用命令行选项 -xx：+PrintCompilation，那么当动态编译运行时将输出一条信息，你可以通过这条消息来验证动态编译是在测试运行前，而不是在运行过程中执行。

通过在同一个 JVM 中将相同的测试运行多次，可以验证测试方法的有效性。第一组结果应该作为“预先执行”的结果而丢弃，如果在剩下的结果中仍然存在不一致的地方，那么就需要进一步对测试进行分析，从而找出结果不可重复的原因。

⊖ JVM 会选择在应用程序线程或后台线程中执行编译过程，不同的选择会对计时结果产生不同的影响。

JVM 会使用不同的后台线程来执行辅助任务。当在单次运行中测试多个不相关的计算密集性操作时，一种好的做法是在不同操作的测试之间插入显式的暂停，从而使 JVM 能够与后台任务保持步调一致，同时将被测试任务的干扰降至最低。（然而，当测量多个相关操作时，例如将相同测试运行多次，如果按照这种方式来排除 JVM 后台任务，那么可能会得出不真实的结果。）

12.3.3 对代码路径的不真实采样

运行时编译器根据收集到的信息对已编译的代码进行优化。JVM 可以与执行过程特定的信息来生成更优的代码，这意味着在编译某个程序的方法 M 时生成的代码，将可能与编译另一个不同程序中的方法 M 时生成的代码不同。在某些情况下，JVM 可能会基于一些只是临时有效的假设进行优化，并在这些假设失效时抛弃已编译的代码[⊖]。

因此，测试程序不仅要大致判断某个典型应用程序的使用模式，还需要尽量覆盖在该应用程序中将执行的代码路径集合。否则，动态编译器可能会针对一个单线程测试程序进行一些专门优化，但只要在真实的应用程序中略微包含一些并行，都会使这些优化不复存在。因此，即便你只是想测试单线程的性能，也应该将单线程的性能测试与多线程的性能测试结合在一起。（在 TimedPutTakeTest 中不会出现这个问题，因为即使在最小的测试用例中都使用了两个线程。）

12.3.4 不真实的竞争程度

并发的应用程序可以交替执行两种不同类型的工作：访问共享数据（例如从共享工作队列中取出下一个任务）以及执行线程本地的计算（例如，执行任务，并假设任务本身不会访问共享数据）。根据两种不同类型工作的相关程度，在应用程序中将出现不同程度的竞争，并表现出不同的性能与可伸缩性。

如果有 N 个线程从共享工作队列中获取任务并执行它们，并且这些任务都是计算密集型的以及运行时间较长（但不会频繁地访问共享数据），那么在这种情况下几乎不存在竞争，吞吐量仅受限于 CPU 资源的可用性。然而，如果任务的生命周期非常短，那么在工作队列上将会存在严重的竞争，此时的吞吐量将受限于同步的开销。

要获得有实际意义的结果，在并发性能测试中应该尽量模拟典型应用程序中的线程本地计算量以及并发协调开销。如果在真实应用程序的各个任务中执行的工作，与测试程序中执行的工作截然不同，那么测试出的性能瓶颈位置将是不准确的。在 11.5 节看到过，对于基于锁的类，例如同步的 Map 实现，在访问锁时是否存在高度的竞争将会对吞吐量产生巨大的影响。本节的测试除了不断地访问 Map 之外没有执行其他操作，因此，虽然只有两个线程，但在所有对 Map 的访问操作中都存在竞争。然而，如果应用程序在每次访问共享数据结构时执行大量的线程本地计算，那么可以极大地降低竞争程度并提供更好的性能。

从这个角度来看，TimedPutTakeTest 对于某些应用程序来说可能是一种不好的模式。由于

⊖ 例如，如果当前已加载的类都没有改写某个方法，那么 JVM 会通过单一调用转换（Monomorphic Call Transformation）将虚拟方法调用转换为直接方法调用。但如果后来加载了一个改写了该方法的类，那么之前已编译的代码将失效。

工作者线程没有执行太多的工作，因此吞吐量将主要受限于线程之间的协调开销，并且对所有通过有界缓存在生产者和消费者之间交换数据的应用程序来说，并不都是这种情况。

12.3.5 无用代码的消除

在编写优秀的基准测试程序（无论是何种语言）时，一个需要面对的挑战就是：优化编译器能找出并消除那些不会对输出结果产生任何影响的无用代码 (Dead Code)。由于基准测试通常不会执行任何计算，因此它们很容易在编译器的优化过程中被消除。在大多数情况下，编译器从程序中删除无用代码都是一种优化措施，但对于基准测试程序来说却是一个大问题，因为这将使得被测试的内容变得更少。如果幸运的话，编译器将删除整个程序中的无用代码，从而得到一份明显虚假的测试数据。但如果不幸运的话，编译器在消除无用代码后将提高程序的执行速度，从而使你做出错误的结论。

对于静态编译语言中的基准测试，编译器在消除无用代码时也存在问题，但要检测出编译器是否消除了测试基准是很容易的，因为可以通过查看机器代码来发现是否缺失了部分程序。但在动态编译语言中，要获得这种信息则更加困难。

在 HotSpot 中，许多基准测试在“-server”模式下都能比在“-client”模式下运行得更好，这不仅是因为“-server”模式的编译器能产生更有效的代码，而且这种模式更易于通过优化消除无用代码。然而，对于将执行一定操作的代码来说，无用代码消除优化却不会去掉它们。在多处理器系统上，无论在正式产品还是测试版本中，都应该选择 -server 模式而不是 -client 模式——只是在测试程序时必须保证它们不会受到无用代码消除优化的影响。

> 要编写有效的性能测试程序，就需要告诉优化器不要将基准测试当作无用代码而优化掉。这就要求在程序中对每个计算结果都要通过某种方式来使用，这种方式不需要同步或者大量的计算。

在 PutTakeTest 中，我们计算了在队列中被添加与删除的所有元素的校验和，但如果在程序中没有用到这个检验和，那么计算校验和的操作仍有可能被优化掉。幸好我们需要通过校验和来验证算法的正确性，然而你也可以通过输出这个值来确保它被用到。但是，你需要避免在运行测试时执行 I/O 操作，以免使运行时间的测试结果产生偏差。

有一个简单的技巧可以避免运算被优化掉而又不会引入过高的开销：即计算某个派生对象中域的散列值，并将它与一个任意值进行比较，例如 System.nanoTime 的当前值，如果二者碰巧相等，那么就输出一个无用并且可被忽略的消息：

```
if (foo.x.hashCode() == System.nanoTime())
    System.out.print(" ");
```

这个比较操作很少会成功，即使成功了，它的唯一作用也就是在输出中插入一个无害的空字符。（在 print 方法中把输出结果缓存起来，并直到调用 println 才真正地执行输出操作，因此即使 hashCode 和 System.nanoTime 的返回值碰巧相等，也不会真正地执行 I/O 操作。）

不仅每个计算结果都应该被使用，而且还应该是不可预测的。否则，一个智能的动态优化

编译器将用预先计算的结果来代替计算过程。虽然在 PutTakeTest 的构造过程中解决了这个问题，但如果测试程序的输入参数为静态数据，那么都会受到这种优化措施的影响。

12.4 其他的测试方法

虽然我们希望一个测试程序能够“找出所有的错误”，但这是一个不切实际的目标。NASA 在测试中投入的资源比任何商业集团投入的都要多（据估计，NASA 每雇佣一名开发人员，就会雇佣 20 名测试人员），但他们生产的代码仍然是存在缺陷的。在一些复杂的程序中，即使再多的测试也无法找出所有的错误。

测试的目标不是更多地发现错误，而是提高代码能按照预期方式工作的可信度。由于找出所有的错误是不现实的，所以质量保证（Quality Assurance，QA）的目标应该是在给定的测试资源下实现最高的可信度。并发程序中的错误通常会比串行程序中更多，因此需要更多的测试才能获得相同的可信度。到目前为止，我们介绍了如何构造有效的单元测试与性能测试。在构建并发类能否表现出正确行为的可信度时，测试是一种非常重要的首选，但并不是唯一可用的 QA 方法。

还有其他一些 QA 方法，它们在找出某些类型的错误时非常高效，而在找出其他类型的错误时则相对低效。通过使用一些补充的测试方法，例如代码审查和静态分析等，可以获得比在使用任何单一方法更多的可信度。

12.4.1 代码审查

正如单元测试和压力测试在查找并发错误时是非常高效和重要的手段，多人参与的代码审查通常是不可替代的。（另一方面，代码审查也不能取代测试。）虽然你可以在设计测试方法时使其能最大限度地发现安全性错误，以及反复地运行这些测试，但同样应该需要有代码编写者之外的其他人来仔细地审查并发代码。即使是并发专家也会有犯错的时候，花一定的时间由其他人来审查代码总是物有所值的。并发专家能够比大多数测试程序更高效地发现一些微妙的竞争问题。（此外，一些平台的问题，例如 JVM 的实现细节或处理器的内存模型等，都会屏蔽一些只有在特定的硬件或软件配置下才会出现的错误。）代码审查还有其他的好处，它不仅能发现错误，通常还能提高描述实现细节的注释的质量，因此将降低后期维护的成本和风险。

12.4.2 静态分析工具

在编写本书时，一些静态分析工具正在迅速地成为正式测试与代码审查的有效补充。静态代码分析是指在进行分析时不需要运行代码，而代码核查工具可以分析类中是否存在一些常见的错误模式。在一些静态分析工具（例如，开源的 FindBugs[⊖]）中包含了许多错误模式检查器，能够检测出多种常见的编码错误，其中许多错误都很容易在测试与代码审查中遗漏。

静态分析工具能生成一个警告列表，其中包含的警告信息必须通过手工方式进行检查，从而确定这些警告是否表示真正的错误。曾经有一些工具（例如 lint）会产生许多伪警告信息，使得开发人员望而却步，但现在的一些工具（例如 FindBugs）已经在这方面有所改进，并且产

⊖ http://findbugs.sourceforge.net。

生的伪警告很少。虽然静态分析工具仍然显得有些原始（尤其在它们与开发工具和开发生命周期的集成过程中），但却足以成为对测试过程的一种有效补充。

在编写本书时，FindBugs 包含的检查器中可以发现以下与并发相关的错误模式，而且一直在不断地增加新的检查器：

不一致的同步。许多对象遵循的同步策略是，使用对象的内置锁来保护所有变量。如果某个域被频繁地访问，但并不是在每次访问时都持有相同的锁，那么这就可能表示没有一致地遵循这个同步策略。

分析工具必须对同步策略进行猜测，因为在 Java 类中并没有正式的同步规范。将来，如果 @GuardedBy 这种标注可以被标准化，那么核查工具就能够解析这些标注，而无须猜测变量与锁之间的关系，从而提高分析的质量。

调用 Thread.run。在 Thread 中实现了 Runnable，因此包含了一个 run 方法。然而，如果直接调用 Thread.run，那么通常是错误的，而应该调用 Thread. start。

未被释放的锁。与内置锁不同的是，执行控制流在退出显式锁（请参见第 13 章）的作用域时，通常不会自动释放它们。标准的做法是在一个 finally 块中释放显式锁，否则，当发生 Exception 事件时，锁仍然处于未被释放的状态。

空的同步块。虽然在 Java 内存模型中，空同步块具有一定的语义，但它们总是被不正确地使用，无论开发人员尝试通过空同步块来解决何种问题，通常都存在一些更好的解决方案。

双重检查加锁。双重检查加锁是一种错误的习惯用法，其初衷是为了降低延迟初始化过程中的同步开销（请参见 16.2.4 节），该用法在读取一个共享的可变域时缺少正确的同步。

在构造函数中启动一个线程。如果在构造函数中启动一个线程，那么将可能带来子类化问题，同时还会导致 this 引用从构造函数中逸出。

通知错误。notify 和 notifyAll 方法都表示，某个对象的状态可能以某种方式发生了变化，并且这种方式将在相关条件队列上被阻塞的线程恢复执行。只有在与条件队列相关的状态发生改变后，才应该调用这些方法。如果在一个同步块中调用了 notify 或 notifyAll，但没有修改任何状态，那么就可能出错（请参见第 14 章）。

条件等待中的错误。当在一个条件队列上等待时，Object.wait 和 Condition.await 方法应该在检查了状态谓词之后（请参见第 14 章），在某个循环中调用，同时需要持有正确的锁。如果在调用 Object.wait 和 Condition.await 方法时没有持有锁，或者不在某个循环中，或者没有检查某些状态谓词，那么通常都是一个错误。

对 Lock 和 Condition 的误用。将 Lock 作为同步块来使用通常是一种错误的用法，正如调用 Condition.wait 而不调用 await（后者能够通过测试被发现，因此在第一次调用它时将抛出 IllegalMonitorStateException）。

在休眠或者等待的同时持有一个锁。如果在调用 Thread.sleep 时持有一个锁，那么将导致其他线程在很长一段时间内无法执行，因此可能导致严重的活跃性问题。如果在调用 Object.wait 或 Condition.await 时持有两个锁，那么也可能导致同样的问题。

自旋循环。如果在代码中除了通过自旋（忙于等待）来检查某个域的值以外不做任何事情，那么将浪费 CPU 时钟周期，并且如果这个域不是 volatile 类型，那么将无法保证这种自

旋过程能结束。当等待某个状态转换发生时，闭锁或条件等待通常是一种更好的技术。

12.4.3 面向方面的测试技术

在编写本书时，面向方面编程 (AOP) 技术在并发领域的应用是非常有限的，因为大多数主流的 AOP 工具还不能支持在同步位置处的“切入点（Pointcut）”。然而，AOP 可以用来确保不变性条件不被破坏，或者与同步策略的某些方面保持一致。例如，在 (Laddad，2003) 中给出了一个示例，其中使用一个方面（Aspect）将所有对非线程安全的 Swing 方法的调用都封装在一个断言中，该断言确保这个调用是在事件线程中执行的。由于不需要修改代码，因此这项技术很容易使用，并且可以发现一些复杂的发布错误和线程封闭错误。

12.4.4 分析与监测工具

大多数商业分析工具都支持线程。这些工具在功能与执行效率上存在着差异，但通常都能给出对程序内部的详细信息（虽然分析工具通常采用侵入式实现，因此可能对程序的执行时序和行为产生极大的影响）。大多数分析工作通常还为每个线程提供了一个时间线显示，并且用颜色来区分不同的线程状态（可运行，由于等待某个锁而阻塞，由于等待 I/O 操作而阻塞等等）。从这些显示信息中可以看出程序对可用 CPU 资源的利用率，以及当程序表现糟糕时，该从何处查找原因。（许多分析工具还声称能够找出哪些锁导致了竞争，但在实际情况中，这些功能与人们期望的加锁行为分析能力之间存在一定的差距。）

内置的 JMX 代理同样提供了一些有限的功能来监测线程的行为。在 ThreadInfo 类中包含了线程的当前状态，并且当线程被阻塞时，它还会包含发生阻塞所在的锁或者条件队列。如果启用了“线程竞争监测（Thread Contention Monitoring）”功能（在默认情况下，为了不影响性能，暂且不启动它），那么在 ThreadInfo 中还会包括线程由于等待一个锁或通知而被阻塞的次数，以及它等待的累计时间。

小结

要测试并发程序的正确性可能非常困难，因为并发程序的许多故障模式都是一些低概率事件，它们对于执行时序、负载情况以及其他难以重现的条件都非常敏感。而且，在测试程序中还会引入额外的同步或执行时序限制，这些因素将掩盖被测试代码中的一些并发问题。要测试并发程序的性能同样非常困难，与使用静态编译语言（例如 C）编写的程序相比，用 Java 编写的程序在测试起来更加困难，因为动态编译、垃圾回收以及自动优化等操作都会影响与时间相关的测试结果。

要想尽可能地发现潜在的错误以及避免它们在正式产品中暴露出来，我们需要将传统的测试技术（要谨慎地避免在这里讨论的各种陷阱）与代码审查和自动化分析工具结合起来，每项技术都可以找出其他技术忽略的问题。